

GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY
Faculty of Engineering
Department of Electrical, Electronics and Telecommunication Engineering

ET4242 – Internet of Things

Practical 05 – Python and MySQL Using Raspberry Pi

Aim**:** To design, implement and evaluate a reliable edge data-logging system on a Raspberry Pi that stores sensor readings in a MySQL-compatible database and supports analytical queries.

Objective**:** To design a normalised schema with least-privilege accounts, build a Python logger that survives database failures, analyse the stored data with SQL, and measure the effect of transaction batching and indexing.

Outcome**:** After completing this practical students would be able to,

a) Design a relational schema with keys, constraints and separate read and write accounts
b) Implement a Python logger with validation, batched inserts and retry after a database failure
c) Analyse stored readings using aggregation, joins and window functions
d) Evaluate experimentally the effect of transaction batching and indexing
e) Critique the design with respect to reliability, security and scalability

Apparatus:

Equipment Required:

- Raspberry Pi 3 or newer with Raspberry Pi OS and network access
- Monitor/keyboard or SSH access to the Raspberry Pi

Components Required:

- DHT11 temperature and humidity sensor

Software Required:

- Python 3 (3.9 or later) with venv, and terminal access
- MariaDB server 10.5 or later (MySQL-compatible)
- mysql-connector-python and matplotlib Python libraries

Theory:

- **Introduction**
  A database on the edge device keeps sensor readings when the network is unavailable, separates sampling from analysis and allows ad-hoc queries. A relational database stores data in tables of rows and columns, and SQL (Structured Query Language) is used to create, insert and retrieve data. MariaDB is a MySQL-compatible server available on Raspberry Pi OS. In this practical a Python program reads a sensor, validates the readings

and writes them reliably to the database; the data is then analysed and the write path is evaluated.

- **Key terms**
  - **Normalisation:** device details are stored once in a `devices` table and each reading refers to it through a foreign key.
  - **Primary key / foreign key:** a primary key uniquely identifies a row; a foreign key links a reading to its device.
  - **Constraint:** `NOT NULL`, `CHECK` and `UNIQUE` rules that keep invalid or duplicate data out of the database.
  - **Data types:** `DECIMAL` stores exact values. Timestamps are stored in UTC as `DATETIME(3)`. The field `recorded_at` is the time the measurement was taken (set by the device), not the time the row was inserted.
  - **Transaction and commit:** InnoDB flushes its log to storage at every commit, which is slow on an SD card. Committing many rows together increases throughput, but rows held in memory are lost if power fails before the commit.
  - **Index:** a B-tree structure that speeds up queries. An index on (`device_id`, `recorded_at`) helps queries that filter on `device_id`, but not on `recorded_at` alone (leftmost-prefix rule), and every index slows inserts.
  - **Store-and-forward:** readings are held in a bounded buffer and the write is retried with increasing delay (exponential back-off) while the database is unavailable.
  - **Idempotent write:** a unique key with `ON DUPLICATE KEY UPDATE` makes a retried insert harmless, so a repeated batch cannot create duplicate rows.
  - **Least privilege:** separate accounts for writing and for reading, each holding only the permissions it needs.
- **Good practice**
  Use a dedicated database user instead of the administrator account, use parameterised SQL statements for all values, keep credentials in a file readable only by its owner (`chmod 600`) and never place them in public repositories or reports. Keep the database on the local Raspberry Pi and do not expose the database port to the internet. Frequent small commits wear SD cards, so stop the logger with Ctrl+C and do not leave benchmarks running unattended. Run destructive statements (`DELETE`, `DROP`) only on your own scratch tables.
- **Preparation before the laboratory**
  Bring the following to the laboratory: (a) an entity-relationship diagram of the `devices` and `sensor_readings` tables showing keys and the relationship; (b) the number of rows generated per day and per year by one device sampling every 5 s, and the storage needed per year assuming about 40 bytes per row (state your assumption); (c) a justified prediction of how the time to insert 5,000 rows changes between committing after every row and committing after every 1,000 rows on an SD card.

Procedure:

1. **Install and Configure the Database**
   Open a terminal on the Raspberry Pi, install and start the database server, and confirm that it listens only on 127.0.0.1:

```
sudo apt update
sudo apt install mariadb-server
sudo systemctl enable --now mariadb
sudo ss -tlnp | grep -E "3306|mariadbd"
```

Open the administrator prompt with `sudo mariadb`. Generate two different strong passwords (for example `openssl rand -base64 18`), replace the placeholders, and run:

```
CREATE DATABASE iot_lab CHARACTER SET utf8mb4;
USE iot_lab;


CREATE TABLE devices (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  device_name   VARCHAR(32)  NOT NULL UNIQUE,
  location      VARCHAR(100),
  sensor_type   VARCHAR(30),
  registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;


CREATE TABLE sensor_readings (
  id               BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  device_id        INT          NOT NULL,
  temperature_c    DECIMAL(5,2) NOT NULL,
  humidity_percent DECIMAL(5,2) NOT NULL,
  recorded_at      DATETIME(3)  NOT NULL,          -- UTC, set by the device
  UNIQUE KEY uq_device_time (device_id, recorded_at),
  CONSTRAINT fk_reading_device FOREIGN KEY (device_id) REFERENCES devices(id),
  CONSTRAINT chk_temp CHECK (temperature_c BETWEEN -40 AND 80),
  CONSTRAINT chk_hum  CHECK (humidity_percent BETWEEN 0 AND 100)
) ENGINE=InnoDB;


CREATE USER 'iot_writer'@'localhost' IDENTIFIED BY '<writer-password>';
CREATE USER 'iot_reader'@'localhost' IDENTIFIED BY '<reader-password>';
GRANT SELECT ON iot_lab.devices          TO 'iot_writer'@'localhost';
GRANT INSERT ON iot_lab.sensor_readings  TO 'iot_writer'@'localhost';
GRANT SELECT ON iot_lab.*                TO 'iot_reader'@'localhost';


INSERT INTO devices (device_name, location, sensor_type)
VALUES ('pi-01', 'Laboratory', 'DHT11 or simulated');
```

Steps:

a. Run `SHOW CREATE TABLE sensor_readings\G` and `SHOW GRANTS FOR 'iot_writer'@'localhost';` as administrator and record the output.

b. Connect as the reader (`mariadb -u iot_reader -p iot_lab`) and attempt `INSERT INTO devices (device_name) VALUES ('x');`. Record the error number and message.

c. As administrator, attempt to insert a reading with `temperature_c = 120` and another with a `device_id` that does not exist. Record both error messages and state which constraint produced each.

2. Set Up the Python Project

Create the project directory and virtual environment, install the libraries and create the credentials file:

```
mkdir -p ~/iot_mysql_lab && cd ~/iot_mysql_lab
python3 -m venv .venv
source .venv/bin/activate
python -m pip install mysql-connector-python matplotlib
nano db.ini          # contents below
chmod 600 db.ini
```

Contents of `db.ini` (never commit this file to a repository):

```
[writer]
host = localhost
user = iot_writer
password = <writer-password>
database = iot_lab

[reader]
host = localhost
user = iot_reader
password = <reader-password>
database = iot_lab
```

Create `db.py`, which returns a connection for the chosen account:

```
import configparser
import os
import mysql.connector

CONFIG_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "db.ini")

def get_connection(role):
    """Return a connection using the credentials of 'writer' or 'reader'."""
    cfg = configparser.ConfigParser(interpolation=None)
    if not cfg.read(CONFIG_PATH):
        raise FileNotFoundError(CONFIG_PATH)
    return mysql.connector.connect(connection_timeout=5, **dict(cfg[role]))
```

Create `sensors.py`. Both classes provide `read()`, which returns `(temperature, humidity)` or `None` when a read fails; the DHT11 fails regularly, so the logger must cope with `None`.

```
import random

class SimulatedSensor:
    """Bounded random walk with occasional failed reads."""
    def __init__(self, fail_rate=0.05):
        self.t, self.h, self.fail_rate = 28.0, 65.0, fail_rate

    def read(self):
        if random.random() < self.fail_rate:
            return None
        self.t = min(max(self.t + random.gauss(0, 0.3), 20.0), 40.0)
        self.h = min(max(self.h + random.gauss(0, 0.8), 30.0), 95.0)
        return round(self.t, 2), round(self.h, 2)

class DHTSensor:
    """DHT11 via the Adafruit library (install as in Practical 03)."""
    def __init__(self, pin_name="D4"):
        import board
        import adafruit_dht
        self.dev = adafruit_dht.DHT11(getattr(board, pin_name))

    def read(self):
        try:
            return float(self.dev.temperature), float(self.dev.humidity)
        except (RuntimeError, TypeError):
            return None
```

3. Part 1 – Fault-Tolerant Data Logger

Create a file named `logger.py`. The program samples the sensor at a fixed rate, validates each reading, places it in a bounded buffer and writes the buffer to the database in one transaction when a batch is complete. If the database is unreachable the readings stay in the buffer and the write is retried after 1 s, 2 s, 4 s and so on up to 60 s. Study the code before running it.

```
import argparse
```

```
import signal
import time
from collections import deque
from datetime import datetime, timezone


from mysql.connector import Error


from db import get_connection
from sensors import DHTSensor, SimulatedSensor


INSERT_SQL = (
    "INSERT INTO sensor_readings "
    "(device_id, temperature_c, humidity_percent, recorded_at) "
    "VALUES (%s, %s, %s, %s) "
    "ON DUPLICATE KEY UPDATE id = id"          # makes retries idempotent
)
running = True


def stop(signum, frame):
    global running
    running = False


def valid(temp, hum):
    return -40 <= temp <= 80 and 0 <= hum <= 100


def get_device_id(conn, name):
    cur = conn.cursor()
    cur.execute("SELECT id FROM devices WHERE device_name = %s", (name,))
    row = cur.fetchone()
    cur.close()
    if row is None:
        raise SystemExit(f"Device '{name}' is not registered in the database.")
    return row[0]


def try_flush(conn, rows):
    """Write rows in a single transaction. Return True on success."""
    try:
        cur = conn.cursor()
        cur.executemany(INSERT_SQL, rows)
        conn.commit()
        cur.close()
        return True
    except Error as err:
        print("  DB error:", err)
        try:
            conn.rollback()
        except Error:
            pass
        return False


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--device", default="pi-01")
    ap.add_argument("--interval", type=float, default=2.0, help="seconds")
    ap.add_argument("--batch", type=int, default=10, help="rows per transaction")
    ap.add_argument("--sensor", choices=["sim", "dht"], default="sim")
    args = ap.parse_args()


    signal.signal(signal.SIGINT, stop)
    signal.signal(signal.SIGTERM, stop)
    sensor = SimulatedSensor() if args.sensor == "sim" else DHTSensor()


    conn = get_connection("writer")
    device_id = get_device_id(conn, args.device)
    buffer = deque(maxlen=10000)        # bounded: oldest rows dropped if full
    stats = {"stored": 0, "failed_reads": 0, "rejected": 0}
    backoff, next_attempt = 1, 0.0
    next_sample = time.monotonic()
    print("Logging started. Press Ctrl+C to stop.")


    while running:
        reading = sensor.read()
        if reading is None:
            stats["failed_reads"] += 1
        elif not valid(*reading):
            stats["rejected"] += 1
```

```
        else:
            now = datetime.now(timezone.utc).replace(tzinfo=None)   # UTC
            buffer.append((device_id, reading[0], reading[1], now))

        if len(buffer) >= args.batch and time.monotonic() >= next_attempt:
            try:
                if not conn.is_connected():
                    conn = get_connection("writer")
                ok = try_flush(conn, list(buffer))
            except Error as err:
                print("  Cannot connect:", err)
                ok = False
            if ok:
                stats["stored"] += len(buffer)
                print(f"Stored {len(buffer)} rows (total {stats['stored']})")
                buffer.clear()
                backoff = 1
            else:
                print(f"  Retry in {backoff} s; {len(buffer)} rows buffered")
                next_attempt = time.monotonic() + backoff
                backoff = min(backoff * 2, 60)

        next_sample += args.interval
        time.sleep(max(0.0, next_sample - time.monotonic()))

    # clean shutdown: one last attempt to write what is still buffered
    if buffer:
        try:
            if try_flush(get_connection("writer"), list(buffer)):
                stats["stored"] += len(buffer)
                buffer.clear()
        except Error as err:
            print("Final flush failed:", err)
    print("Summary:", stats, "| unsent:", len(buffer))

if __name__ == "__main__":
    main()
```

Steps:

a. Register the device with `INSERT INTO devices (device_name, location, sensor_type) VALUES ('pi-01', 'Laboratory', 'DHT11 or simulated');` as administrator.

b. Activate the virtual environment and run `python logger.py --device pi-01 --interval 1 --batch 10`.

c. Let it run for at least two minutes (at least 100 samples), then stop it with Ctrl+C and record the summary line.

d. Check the stored row count with `SELECT COUNT(*) FROM sensor_readings;`. Explain any difference between (run time ÷ interval) and the stored count using the summary statistics.

e. If a DHT11 is available, repeat with `--sensor dht` and compare the proportion of failed reads with the simulated sensor.

4. Part 2 – Fault Injection and Data Integrity

Complete the following tasks and demonstrate the results to the instructor.

a. Start the logger with `--interval 1 --batch 10`.

b. In a second terminal run `sudo systemctl stop mariadb`, wait about 30 s while observing the retry messages, then run `sudo systemctl start mariadb`.

c. Allow the logger to write the buffered data, then stop it with Ctrl+C.

d. Find the largest gaps between consecutive readings with the query below (adjust `device_id` if needed):

```
SELECT * FROM (
  SELECT recorded_at,
         TIMESTAMPDIFF(MICROSECOND,
             LAG(recorded_at) OVER (ORDER BY recorded_at), recorded_at) / 1e6 AS
gap_s
  FROM sensor_readings WHERE device_id = 1
) t
ORDER BY gap_s DESC LIMIT 5;
```

Report whether any data was lost during the outage, the largest gap observed and why it is larger than the sampling interval. Then check that no duplicate (`device_id`, `recorded_at`) pairs exist and explain which constraint guarantees this.

5. Part 3 – Time-Series Analysis with SQL

Connect as the read-only account (`mariadb -u iot_reader -p iot_lab`) and run the following queries on the data collected in Parts 1 and 2. Query (a) gives per-minute aggregates using a join:

```
SELECT d.device_name,
       DATE_FORMAT(r.recorded_at, '%Y-%m-%d %H:%i:00') AS minute,
       COUNT(*)                          AS samples,
       ROUND(AVG(r.temperature_c), 2)    AS avg_temp,
       MIN(r.temperature_c)              AS min_temp,
       MAX(r.temperature_c)              AS max_temp
FROM sensor_readings r JOIN devices d ON d.id = r.device_id
GROUP BY d.device_name, minute
ORDER BY minute;
```

Query (b) gives a five-sample moving average using a window function:

```
SELECT recorded_at, temperature_c,
       ROUND(AVG(temperature_c) OVER (
             PARTITION BY device_id ORDER BY recorded_at
             ROWS BETWEEN 4 PRECEDING AND CURRENT ROW), 2) AS moving_avg
FROM sensor_readings
ORDER BY recorded_at DESC LIMIT 20;
```

Write the following queries yourself and record each query, the first rows of its output and a one-sentence interpretation:

a. Outliers: all readings whose temperature differs from the device mean by more than two standard deviations (hint: a subquery using `AVG` and `STDDEV`).
b. Availability: for each device and hour, the percentage of expected samples actually stored (expected = 3600 ÷ sampling interval).
c. Rate of change: using `LAG`, list consecutive readings where the temperature changed by more than 1 °C, and comment on whether these are sensor faults or real events.

6. Part 4 – Performance Evaluation

Effect of commit strategy

As administrator, create a scratch table so that benchmark rows do not mix with real data, and allow the writer account to use it:

```
CREATE TABLE iot_lab.bench_readings LIKE iot_lab.sensor_readings;
GRANT INSERT, DELETE ON iot_lab.bench_readings TO 'iot_writer'@'localhost';
```

Create `bench_insert.py`, which inserts the same 5,000 rows using different transaction sizes and reports the throughput:

```
import time
from datetime import datetime, timedelta
```

```
from db import get_connection

N = 5000
SQL = ("INSERT INTO bench_readings "
       "(device_id, temperature_c, humidity_percent, recorded_at) "
       "VALUES (%s, %s, %s, %s)")
base = datetime(2026, 1, 1)
rows = [(1, 25 + (i % 100) / 10, 60 + (i % 200) / 10,
         base + timedelta(seconds=i)) for i in range(N)]

def run(batch):
    conn = get_connection("writer")
    cur = conn.cursor()
    cur.execute("DELETE FROM bench_readings")
    conn.commit()
    t0 = time.perf_counter()
    for i in range(0, N, batch):
        cur.executemany(SQL, rows[i:i + batch])
        conn.commit()                      # one commit per batch
    elapsed = time.perf_counter() - t0
    cur.close()
    conn.close()
    return elapsed

for batch in (1, 10, 100, 1000):
    t = run(batch)
    print(f"batch={batch:5d}  time={t:7.2f} s  rate={N / t:9.0f} rows/s")
```

Steps:

a. Run the script three times and record the mean throughput for each batch size in Table 1.
b. Compare the results with your preparation prediction and explain the trend in terms of the commit cost on an SD card.
c. For the batch size you would choose at a 1 s sampling interval, state the maximum number of readings and the maximum time span that could be lost at an unexpected power failure.

Effect of an index

As administrator, load 200,000 rows into the scratch table using the SEQUENCE engine and compare the query plans with `ANALYZE`, which executes the query and reports the rows actually examined (`r_rows`):

```
USE iot_lab;
DELETE FROM bench_readings;
INSERT INTO bench_readings (device_id, temperature_c, humidity_percent, recorded_at)
SELECT 1, 20 + (seq % 200) / 10, 50 + (seq % 400) / 10,
       TIMESTAMP('2026-01-01') + INTERVAL seq SECOND
FROM seq_1_to_200000;

-- Q1: time range only
ANALYZE SELECT COUNT(*), AVG(temperature_c) FROM bench_readings
WHERE recorded_at BETWEEN '2026-01-02 00:00:00' AND '2026-01-02 01:00:00';

-- Q2: device and time range
ANALYZE SELECT COUNT(*), AVG(temperature_c) FROM bench_readings
WHERE device_id = 1
  AND recorded_at BETWEEN '2026-01-02 00:00:00' AND '2026-01-02 01:00:00';

ALTER TABLE bench_readings ADD INDEX idx_time (recorded_at);
-- now repeat Q1 and Q2
```

Steps:

a. For Q1 and Q2, before and after adding `idx_time`, record the `key` chosen, `rows`, `r_rows` and the execution time reported by the client in Table 2.

b. Explain why Q1 could not use the existing composite index efficiently but Q2 could.
c. Measure the insert throughput (batch size 1000) again with `idx_time` present and state the cost of the additional index.

7. Part 5 – Visualisation and Extensions

Write `plot_readings.py` which, using the read-only account, retrieves the last 30 minutes of data for a device given on the command line and saves a figure as a PNG file. The program must:

a. use a parameterised query and close the connection correctly;
b. plot temperature and humidity against time with labelled axes and units;
c. compute and overlay a five-point moving average and mark readings more than two standard deviations from the mean;
d. run without a display (call `matplotlib.use("Agg")` before importing `pyplot`) and print the number of rows plotted.

Extension tasks (complete at least two):

a. Multi-device operation: register a second device and run two logger instances at the same time. Show per-device aggregates and explain how the schema supports more devices without modification.
b. Retention and down-sampling: create a table `readings_hourly` and a maintenance job (cron or the MariaDB event scheduler) that aggregates data older than 24 hours and deletes the raw rows, using a third account with only the privileges it needs.
c. Backup and recovery: back up with `mysqldump --single-transaction`, restore into a new database `iot_lab_restore` and prove equality using `CHECKSUM TABLE`.
d. MQTT pipeline: use the broker and clients from Practical 04 to publish readings, and write a subscriber that validates them and inserts them using the idempotent insert. State the delivery guarantee provided end to end.
e. Persistent spool: replace the in-memory buffer with a local SQLite file and show that no readings are lost when the logger is terminated abruptly (`kill -9`) during an outage.

Results and Observations:

Record the following in your observation sheet.

- Output of the listening-port check and the error messages from the verification tests (Procedure 1):
- Logger summary line and stored row count (Part 1):
- Largest data gap during the outage and number of duplicates found (Part 2):
- Results of the SQL analysis queries (Part 3):
- Errors encountered and how they were resolved:

Table 1 – Insert throughput (mean of three runs)

Batch size	Time (s)	Rows/s	Speed-up vs batch = 1
1			1.0
10			
100			
1000			

Table 2 – Query plans before and after indexing

Query	Index state	key used	rows	r_rows	Time
Q1	without idx_time				
Q1	with idx_time				
Q2	without idx_time				
Q2	with idx_time				

Questions:

1. Justify the two-table design against a single table that stores the device name in every reading. State one situation in which denormalisation would be acceptable.
2. Why is `DECIMAL(5,2)` used instead of `FLOAT`? Why is `recorded_at` supplied by the device in UTC rather than generated by the server at insertion?
3. Explain your results in Table 1. How would they differ on an SSD? Which MariaDB setting trades durability for throughput, and how does relaxing it differ from batching rows in the application?
4. Explain why the composite index did not help Q1 but did help Q2, and why adding `idx_time` slows down inserts.
5. The default cursor of `mysql-connector-python` escapes parameters on the client, whereas a prepared-statement cursor sends them separately to the server. Explain how each protects against SQL injection and why string concatenation does not.
6. Explain why `ON DUPLICATE KEY UPDATE id = id` is needed in the logger. Describe the failure sequence that would otherwise create duplicate rows or block the logger permanently.
7. List three weaknesses of the in-memory store-and-forward design (for example power loss, buffer overflow, long outages) and propose an improvement for each.
8. A deployment has 10,000 devices sampling at 1 Hz. Calculate the rows per day and the approximate yearly storage, and discuss whether a single MariaDB instance on a Raspberry Pi is appropriate compared with partitioning or a dedicated time-series database.

Performance Evaluation

Experiment: Python and MySQL Using Raspberry Pi

	Evaluation Aspect	Marks
1	Preparation	
2	Neatness of Work	
3	Familiarity with Lab Equipment	
4	Completion of Work	
5	Capability	
6	Accuracy of Readings/ Observations	
7	Answers given to Questions	
8	Discipline	
	Total	

Marks are awarded on a 0-10 scale for each aspect

Excellent	Very Good	Good	Fair	Poor	Very Poor
10	9 – 8	7 – 6	5 – 4	3 – 2	1 – 0

Name of the Evaluator: ……

Signature: ……

Date: ……